

Proiect ASO 2025

Etapa 2: Agent AI Administrator de Sistem

Student: Vlad Roatis

Grupa: 30641

Data: 5 decembrie 2025

1. Introducere

- În această etapă a proiectului, am migrat aplicația monolitică dezvoltată în Etapa 1 către o arhitectură de microservicii bazată pe containere Docker. Scopul principal a fost izolarea componentelor sistemului pentru a simula un mediu de producție real, unde agentul, serverul de unelte (MCP) și modelul LLM rulează pe instanțe separate, comunicând prin rețea.
- De asemenea, am implementat cerința de Bonus, adăugând un mecanism de autentificare bazat pe API Key, astfel încât serverul MCP să răspundă doar solicitărilor venite de la un agent autorizat.

1.1 Obiective Etapa 2

- Separarea componentelor în containere Docker distincte: Ollama, MCP Server, ADK Agent.
- Configurarea comunicării între containere folosind un bridge network în Docker.
- Trecerea serverului MCP de la protocolul stdio la protocolul HTTP (SSE).
- Bonus: Implementarea securității prin validarea unei chei de acces (API Key) la fiecare apel de funcție.

2. Arhitectura Sistemului

Sistemul este compus din trei servicii definite în docker-compose.yml:

- Serviciul Ollama (aso_ollama):
 - Rulează imaginea oficială ollama/ollama.
 - Expune portul 11434 pentru API-ul LLM.
 - Utilizează un volum persistent pentru a stoca modelele descărcate (llama3.2:3b).
- Serviciul MCP Server (aso_mcp):
 - Rulează într-un container Python custom.
 - Expune un server HTTP pe portul 7000.
 - Are acces la admin_folder prin montarea unui volum, permițând manipularea fișierelor de pe sistemul gazdă.
- Verifică prezența cheii de securitate MCP_API_KEY.

- Serviciul ADK Agent (aso_adk_web):
- Găzduiește agentul inteligent și interfața web de chat.
- Se conectează la Ollama și la MCP Server folosind numele serviciilor din rețeaua Docker (http://ollama:11434 și http://mcp-server:7000).
- Este configurat să trimită automat cheia de securitate la fiecare interogare.

3. Implementare Docker

- 3.1 Dockerfiles
- Am creat fișiere Dockerfile specifice pentru fiecare componentă care necesită cod personalizat.
- Dockerfile.mcp (pentru Server): Instalează biblioteca fastmcp și uvicorn. Setează variabilele de mediu pentru a rula serverul în mod HTTP (MCP_TRANSPORT=http) și expune portul 7000.
- Dockerfile.adk (pentru Agent): Copiază codul sursă al agentului (src/) și configurațiile. Instalează pachetele necesare (google-adk, litellm) și expune portul 8000 pentru interfața web.

3. Structura Proiectului

```

proiect_aso/
├── admin_folder/           #folderul partajat
│   ├── abcde
│   └── system_info.txt
├── config/
├── src/
│   ├── __init__.py
│   ├── agent.py           #logica agent+api key
│   └── mcp_server/        #componenta server
│       ├── mcp_server.py
│       ├── Dockerfile.mcp
│       └── requirements.txt
├── Dockerfile.adk
├── Dockerfile.ollama
├── docker-compose.yml     #Docker
└── requirements.txt

```

5. Rulare și Testare

5.1 Pornirea sistemului

Sistemul se pornește folosind comanda:

```
docker-compose up --build
```

Aceasta construiește imaginile și pornește containerele în ordinea dependențelor.

6 Exemple de Interacțiune

hello

list_directory

list_directory

Hello! I'm the system administration agent. How can I assist you today? Do you have a question about the file system or would you like to create a new directory/file?

what files do i have?

list_directory

list_directory

You have two files in the current directory: `abcde` and `system_info.txt`. The `abcde` is a directory, while the `system_info.txt` is a file with 44 bytes of content. Would you like to see the contents of `system_info.txt` or list the files again?

what else can you do?

Get the content of the file `abcde/mamama.txt`

get_file_content

get_file_content

The contents of '`abcde/mamama.txt`' are:
fcsb e steaua

Agent cu `api_key` gresita:

hello! what files do i have?

list_directory

list_directory

ACCES DENIED